

Testez et améliorez une application existante

Qu'allez-vous apprendre dans ce projet ?

Ce projet marque le début de votre parcours pour devenir expert en développement logiciel. À ce stade, vous n'avez pas encore appris de compétences spécifiques dans votre formation.

Cependant, vous devez vérifier avec votre mentor vos compétences actuelles en développement. Ce projet est conçu pour vous permettre de pratiquer les concepts et techniques nécessaires.

Dans ce projet, vous allez démontrer votre niveau en développement full-stack en intégrant une nouvelle fonctionnalité dans une application existante et en mettant en place des tests automatisés pour garantir sa qualité et sa stabilité.

Vous manipulez à la fois Angular, un framework front-end, et Java, un langage back-end, en appliquant les bonnes pratiques du développement logiciel. Vous explorerez aussi la pyramide des tests, la logique du TDD (Test-Driven Development) et l'utilisation de frameworks de test comme Jest, JUnit, Mockito ou Cypress.

Aucune connaissance préalable en Angular ou en Java n'est requise pour ce parcours. Mais lors de ce projet, il faudra vous familiariser avec les bases de ces deux technologies pour le réussir. Dans ce cadre, si vous n'avez aucune connaissance préalable en Angular ou en Java, il est nécessaire de suivre les sections Remise à niveau afin d'obtenir les bases des connaissances sur ces deux technologies et sur les frameworks de test associés.

Après ce projet, vous n'êtes pas du tout obligé de monter en compétences sur Angular ni sur Java ni de continuer de les utiliser. Ils sont ici utilisés pour vous fournir du code concret mais le but de ce projet est de vous donner l'occasion d'approfondir vos compétences en développement en général.

En quoi ces compétences sont-elles importantes pour votre carrière ?

Savoir développer une application du front-end au back-end, tout en assurant sa qualité par les tests et le débogage, c'est essentiel au niveau senior dans le monde du développement. Ces compétences vous rendent rapidement opérationnel sur des projets complexes, vous permettant d'assurer maintenabilité, fiabilité et évolutivité. Vous saurez aussi identifier vos lacunes et apprendre à progresser efficacement, une compétence clé pour les profils senior. Ce projet est découpé en 4 activités, dont 2 restent optionnelles si vous avez déjà assez d'expérience en développement d'applications.

Remise à niveau – Bases du développement full-stack Java et Angular (optionnel) : si vous n'êtes pas encore à l'aise avec le développement des applications full-stack et/ou vous ne connaissez pas encore Java ni Angular, vous suivrez dans cette section des cours de développement pour vous familiariser avec les outils.

Exercice – Améliorez et ajoutez des fonctionnalités : vous commencerez par améliorer une fonctionnalité en débogant, puis vous implémenterez et testerez une nouvelle fonctionnalité.

Ce codebase est en Java et Angular. Néanmoins, pas d'inquiétude si vous n'avez pas encore de connaissances Java / Angular : le codebase reste simple et vous serez guidé.

Remise à niveau – Testez votre code (optionnel) : si vous n'êtes pas encore à l'aise avec les tests d'applications full-stack, vous suivrez dans cette section des cours dédiés à cette thématique.

Vous vous familiarisez également avec les frameworks de test sur Java et Angular : JUnit, Mockito, Jest et Cypress.

Exercice – Effectuez des tests : vous implémentez des tests unitaires, d'intégration et end-to-end pour assurer la maintenance du codebase.

Vous terminerez chaque exercice en complétant la fiche d'autoévaluation qui servira de base de discussion et de bilan avec votre mentor.

À l'issue de ce projet, vous aurez une session de bilan avec votre mentor pour discuter de votre projet.

Cela vous assurera que vous êtes sur la bonne voie avant de passer à la suite.

Remise à niveau : Bases du développement full-stack

Avant d'aborder les exercices de ce projet, il est essentiel de vous assurer que vous possédez les bases du développement front-end, back-end, ainsi que les notions fondamentales d'outillage (Git, terminal, etc.).

De plus, si vous ne connaissez pas ou peu Angular et Java, il vous faudra apprendre les bases des deux technologies car ce sont celles mises en œuvre dans ce projet :

Angular, un framework front-end utilisé dans des milliers d'entreprises pour développer des applications web dynamiques et structurées ;

Java, un langage back-end utilisé dans l'industrie logicielle pour créer des applications robustes et maintenables avec des données bien sécurisées.

Les compétences en développement sont indispensables pour implémenter des fonctionnalités de façon fluide, comprendre une base de code existante, et produire du code maintenable. Elles vous permettront de progresser efficacement sur les étapes suivantes du projet.

Sur quelles parties devez-vous vous focaliser ?

Cette section est optionnelle, mais fortement recommandée si vous avez un doute sur votre maîtrise des outils ou technologies de développement.

Prêt à résoudre l'exercice ?

Dans cet exercice, vous reprenez un projet Java/Angular d'entreprise à travers une application web en cours de développement, EtuBibliothèque.

EtuBibliothèque est une application qui permet de gérer les étudiants abonnés à une bibliothèque. Elle est découpée en 2 parties :

un back-end (voici le dépôt GitHub du back-end)

un front-end (voici le dépôt GitHub du front-end)

Le code de départ permet uniquement d'enregistrer les agents de la bibliothèque. Il faut ajouter de nouvelles fonctionnalités. C'est ici que vous intervenez !

En somme, votre mission consiste à :

analyser les starters code existants pour le back-end et le front-end ;

lancer le front-end et le back-end et vérifier que l'application existante fonctionne ;

ajouter de nouvelles fonctionnalités pour enrichir l'application existante ;

ajouter des tests unitaires et d'intégration pour s'assurer de la bonne qualité du code.

Il n'y a pas de spécifications ni de maquettes pour ce projet. Vous devez simplement suivre les étapes fournies.

Le but des exercices est d'implémenter et de tester le code front et back en suivant les bonnes pratiques de développement.

Cet exercice est entièrement guidé.

Vous pouvez suivre les étapes ci-dessous.

Étapes 1 : Analyser le code existant

Vous allez explorer les starters code fournis et lire les codes back-end et front-end. Cette étape est cruciale pour comprendre l'architecture actuelle du projet. Essayez de comprendre l'organisation des fichiers, les classes principales et les interactions front/back.

Prérequis

installé les technologies utilisées :

Java 21 et Maven 3.9.3

Angular 19

Docker, Docker Compose et Docker Desktop sur votre environnement de travail local

cloné et ouvert les starters code dans votre environnement de travail ;

lancé l'application en local :

le back-end pour confirmer qu'elle démarre avec la base de données

le front-end pour confirmer qu'elle démarre

Résultat attendu

Compréhension des starters code front et back et leurs interactions.

Les applications back-end et front-end démarrent normalement et communiquent entre elles.

On arrive à créer un utilisateur agent de la bibliothèque.

Recommandations

D'abord, suivez les instructions dans le fichier README.md du dépôt back-end afin de lancer le projet.

Assurez-vous d'avoir installé Docker, Docker Compose et Docker Desktop sur votre environnement de travail local.

Démarrez le code back-end :

Ouvrez le fichiersrc/main/resources/application.yml. Les credentials d'accès à la base de données sont masqués par des variables d'environnement. Ces variables d'environnement sont définies dans le fichier .env

Ouvrez le fichiercompose.yml. Il définit la base de données MySQL qui est démarrée avec l'application et qui contient les données nécessaires au fonctionnement du back-end.

Parcourez les configurations et les couches de l'application

Lancez le back-end. Pour cela, veuillez lire la documentation README.md du projet, section

Démarrage du back-end.

Démarrez le code front-end :

Installez les dépendances du projet avec la commande :npm install

Parcourez les fichiers contenus dans le répertoiresrc/apppour comprendre les configurations.

Lancez le front-end (npm run start)

Testez l'enregistrement d'un agent de la bibliothèque :

Lorsque le front-end est lancé, ouvrez votre navigateur et saisissez l'adresse :

<http://localhost:4200/register>

Remplissez le formulaire d'enregistrement d'un nouvel utilisateur et valider.

Lisez les logs du back-end. Vous devez voir la trace d'insertion d'un nouvel utilisateur dans la table des utilisateurs.

Vous pouvez vous connecter aussi à la base de données via Docker Desktop et consulter la tableuser.

Un nouvel utilisateur doit apparaître dans cette table de données.

Pour rappel, dans le README.md du projet, il est indiqué pas à pas comment se connecter à la base de données et consulter le contenu de la tableuser.

Prenez des notes lors de vos explorations des starters code, en particulier des endroits où vous avez besoin de creuser plus profondément.

Points de vigilance

Ne modifiez pas encore le code à cette étape.

Ne vous arrêtez pas seulement à la structure des fichiers : regardez aussi le contenu logique.

Vérifiez le comportement visuel (même si c'est encore simple).

Outils

Les IDE :

VS Code

IntelliJ

Angular CLI

Java 21

Maven 3.9.3

Docker, Docker Compose et Docker Desktop

Ressources

Documentation sur l'installation de Docker, Docker Compose et Docker Desktop :

<https://docs.docker.com/desktop/>

<https://docs.docker.com/engine/install/>

Etape 2 : Corriger l'API d'authentification des utilisateurs côté Backend

Corrigez le service d'implémentation de l'API d'authentification des utilisateurs dans la partie back-end, en suivant le modèle de l'existant.

Prérequis

exploré et analysé les starters code ;

identifié l'endpoint actuel d'authentification/api/login ;

compris la structure des classes : controller, service, etc...

Résultat attendu

L'API d'authentification fonctionne bien et retourne un token JWT lorsque l'authentification est réussie.

Recommandations

L'API d'authentification d'un utilisateur/api/login ne fonctionne pas et rencontre quelques bugs.

Déboguez le code existant :

Mettez des points d'arrêt dans les différents fichiers pour comprendre les appels entre les différentes classes et identifier la cause du bug.

Assurez-vous que l'utilisateur envoie un login et un mot de passe valides.

Relisez le service d'authentification UserService.java, méthode 'login', puis ajoutez les points d'arrêt pour déboguer et identifier les incohérences et les inexactitudes à corriger.

Apportez une correction appropriée pour que cette API fonctionne normalement.

Pour rappel, cette API doit retourner un token JWT lorsque l'authentification réussit.

Pensez à implémenter le service JWTService.java qui est chargé de délivrer un token JWT.

Testez le fonctionnement de cette route avec Postman.

Points de vigilance

Respectez les bonnes pratiques : découpage de code, respect des couches, etc...

N'ignorez pas les conventions de nommage.

Si vous n'avez pas encore utilisé Postman, suivez ces étapes :

EDI : VS Code, IntelliJ

Postman

Entity To DTO Conversion for a Spring REST API | Baeldung

Comment importer une collection Postman (documentation officielle)

Tutorial: Debug your first Java application | IntelliJ IDEA

Debug code | IntelliJ IDEA Documentation

Etape 3 : Implémenter l'interface d'authentification côté Front-End

Ajoutez une interface utilisateur qui permet d'authentifier un utilisateur et de consommer l'API améliorée précédemment.

Prérequis

Avoir :

testé l'endpoint back-end `/api/login` avec succès ;

compris le framework front Angular.

Résultat attendu

L'utilisateur peut déclencher la fonctionnalité d'authentification depuis l'interface.

Le front-end communique avec le back-end avec succès pour authentifier un utilisateur.

Recommandations

Dans l'application front-end, vous développerez un écran d'authentification (nouveau composant login) et lui associerez une route.

Cet écran doit rester simple, seulement avec des boutons et champs obligatoires pour permettre les fonctionnalités de login et mot de passe.

Réalisez un service pour que le nouvel écran appelle l'API `/api/login` développée précédemment.

En vous inspirant de l'existant, créez les `model`, `composant`, `service`, etc., nécessaires pour implémenter avec succès cette nouvelle fonctionnalité.

Testez le nouvel écran en appelant le back-end et vérifiez que tout fonctionne bien. Vérifiez que vous recevez un token d'authentification.

Respectez les échanges d'information entre le front-end et le back-end via des DTO.

Points de vigilance

Assurez-vous que les erreurs serveur s'affichent.

Gérez les états : chargement, erreur, succès.

Ressources

Le cours Débutez avec Angular, Partie 5 :

le chapitre Partagez des données avec les Services

le chapitre Passez en SPA avec le routing

Angular Services Guide (pour Angular v17, mais toujours pertinent au code du front en v19)

Etape 4 : Ajouter de nouvelles fonctionnalités côtés back-end

Ajoutez des APIs CRUD (Create, Read, Update, Delete en anglais) pour gérer les étudiants de la bibliothèque :

Ajouter un étudiant

Consulter la liste des étudiants

Consulter les informations détaillées d'un étudiant

Modifier un étudiant

Supprimer un étudiant

Prérequis

Avoir :

terminé l'implémentation front/back de l'authentification avec un token JWT.

Résultat attendu

Toutes les nouvelles APIs fonctionnent normalement pour gérer les étudiants.

Recommandations

À partir de cette étape, vous devez implémenter les différentes routes pour ajouter, consulter, modifier et supprimer un étudiant (c'est-à-dire effectuer les actions CRUD).

Découpez bien le code en suivant le modèle en couches :

les entrées/sorties sont gérées par le controller ;

les traitements sont faits dans le service ;

l'accès aux données est fait par des repository.

Pensez à utiliser les DTO. Les entités ne doivent pas apparaître dans les controllers.

Toutes ces APIs doivent être sécurisées. L'utilisateur doit être authentifié (utiliser un Bearer Token dans le header) avant de pouvoir exploiter ces APIs.

Dès qu'une API est implémentée, testez le fonctionnement avec Postman.

Points de vigilance

Les APIs CRUD sont accessibles uniquement aux utilisateurs authentifiés (configuration Spring Security, etc....).

Si vous rencontrez des erreurs en implémentant votre code Java, suivez des méthodes de débogage.

EDIs : VS Code, IntelliJ

Ressources

Entity To DTO Conversion for a Spring REST API | Baeldung

Débogage de code avec IntelliJ

Tutorial: Debug your first Java application | IntelliJ IDEA

Debug code | IntelliJ IDEA Documentation

Etape 5 : Implémenter les écrans côté front-end

Développez les écrans du front-end pour consommer les APIs codées précédemment.

Prérequis

terminé l'implémentation des APIs CRUD pour gérer les étudiants ;

lu la documentation sur Angular.

Résultat attendu

Tous les écrans nécessaires pour gérer les étudiants sont codés.

Le front-end consomme les APIs CRUD du back-end.

L'utilisateur peut effectuer les actions décrites ci-dessous.

Recommandations

Dans l'application front-end, développez les écrans pour les opérations CRUD :

ajouter un nouvel étudiant

consulter les étudiants

consulter le détail d'un étudiant

modifier un étudiant

supprimer un étudiant

Pour rappel, il n'est pas nécessaire de coder de beaux écrans.

Réalisez les services Angular pour appeler les APIs développées précédemment.

Faites en sorte que les informations envoyées depuis le front-end correspondent aux DTO attendus du côté du back-end.

Testez les écrans en appelant le back-end et vérifiez que tout fonctionne bien.

Points de vigilance

Protégez les routes d'accès aux écrans des étudiants par l'utilisation de Guard Angular.

Si l'utilisateur n'est pas connecté, il ne peut pas créer, consulter, modifier et supprimer des étudiants.

Outils

EDIs : VS Code, IntelliJ

Ressources

Le cours Débutez avec Angular, Partie 5 :

le chapitre Partagez des données avec les Services
le chapitre Passez en SPA avec le routing
Angular Services Guide